

A Hacker's Guide to ELIZA

Copyright © 2026 Jeff Shrager (jshrager@gmail.com). All rights reserved. (See copyright details at the end of this document.)

For the curious visitor who wants to read the original 1965 MAD-SLIP source of the world's most famous chatbot — the code itself, not the legend

Before You Arrive: What Is ELIZA?

ELIZA is a conversation program written by Joseph Weizenbaum at MIT between 1964 and 1966, running under CTSS — the first general-purpose time-sharing system — on an IBM 7094. Its most famous script, DOCTOR, parodies a Rogerian psychotherapist: you type "MY BOYFRIEND MADE ME COME HERE" and it answers with your own words turned back at you. Weizenbaum built it partly to show how cheaply the *impression* of understanding could be manufactured; to his lasting dismay, people confided in it anyway.

A note on what you're reading. For over half a century the actual source was missing and widely misremembered: most people assumed ELIZA was a Lisp program, because every famous descendant was. In 2021, Jeff Shrager's ELIZAGEN archaeology project, working in MIT's Institute Archives, opened a folder labeled "Computer Conversations" in Weizenbaum's papers and found a complete printout: the ELIZA core in **MAD-SLIP**, support functions, and an early version of the DOCTOR script attached. Weizenbaum's estate released it under CC0. One thing to fix in mind before you start reading: this listing dates to around 1965 and **predates the January 1966 CACM paper**. You are not reading the system the paper describes — you are reading an earlier developmental stage of it. Where the code and the paper differ (and they do, in interesting places), the listing is a snapshot of ELIZA partway through its evolution, not a contradiction of the published account.

MAD ("Michigan Algorithm Decoder") is an ALGOL-58-family language; SLIP ("Symmetric List Processor") is Weizenbaum's own doubly-linked-list library, which he wrote years before ELIZA. The transcription you'll read (by Anthony Hay, with annotations) covers six MAD routines totaling a few hundred lines: `CHANGE`, `TPRINT`, `LPRINT`, `TESTS`, `DOCBCD`, and the main program, `ELIZA`.

That's the first surprise of the visit: the whole celebrated artifact is *smaller than this guide*.

The Map: Six Districts

[The Charter]	the DOCTOR script — the personality, as pure data
[The Substrate]	SLIP — lists, sequence readers, HASH, and the pattern matcher ELIZA merely borrows
[City Hall]	the ELIZA main program: load script, scan input, pick a keyword, answer, repeat
[The Customs Office]	TESTS — full-word matching and on-the-fly word substitution (MY → YOUR)
[The Memory Annex]	MYTRAN / MYLIST / LIMIT — the "certain counting mechanism," demystified
[The Classroom]	CHANGE and friends — the part nobody knew about: ELIZA was built to be taught

District 1: The Charter — the script

ELIZA the program has no opinions, no therapy, no English. All of that lives in a script — a file of parenthesized lists that the program reads from tape at startup ("WHICH SCRIPT DO YOU WISH TO PLAY"). A script entry bundles a **keyword** with optional machinery:

```
(MY = YOUR 2
  ((0 YOUR 0 (/FAMILY) 0)
   (TELL ME MORE ABOUT YOUR FAMILY) ... )
...)
```

Reading left to right: the keyword `MY`; an `=` substitution (whenever the user says MY, rewrite it as YOUR before answering); a precedence number (2); then pairs of **decomposition rules** — patterns like `(0 YOUR 0)`, where 0 means "any number of words" — and **reassembly rules** that rebuild a reply from the matched fragments (`3` in a reassembly means "insert the third matched fragment"). Special entries include `NONE` (stock replies for when nothing matches), `MEMORY` (see District 5), and cross-references like `(HOW (=WHAT))` — HOW has no rules of its own; it borrows WHAT's.

★ DON'T MISS

The script is parenthesized exactly like Lisp S-expressions — which is almost certainly why

the world spent fifty years believing ELIZA was Lisp. The Lisp-looking part of ELIZA is the data. The program that interprets it is ALGOL-family code full of GOTOs.

District 2: The Substrate — SLIP

ELIZA is a thin program standing on a thick library — and here, unusually, the author of the library and the author of the program are the same person, with the library doing most of the lifting. The idioms you'll see on nearly every line:

- `LIST.(X)` — make X a new empty list; `MTLIST.` empties one; `IRALST.` reclaims one
- `NEWTOP.` / `NEWBOT.` / `POPTOP.` / `POPBOT.` — push and pop at either end (the lists are doubly linked — that's the "Symmetric")
- `S=SEQDR.(L)` then `SEQLR.(S,F)` — a **sequence reader**: a cursor that walks a list element by element, with flag `F` telling you whether you got a datum, a sublist, or ran off the end
- `HASH.(WORD,N)` — hash a word into 2^N buckets
- `YMATCH.` and `ASSMBL.` — match a decomposition pattern against text; assemble a reply from a reassembly rule and matched fragments

★ DON'T MISS

Those last two. The single most famous thing ELIZA does — pattern-match

`(0 YOUR 0)` against a sentence and rebuild fragments into a reply — is not in this listing at all. `YMATCH` and `ASSMBL` are SLIP library calls, and each is invoked on exactly **one line** of the main program. The legendary mechanism is two function calls; everything around it is bookkeeping. (The SLIP library itself, in MAD and FAP assembly, was reconstructed separately during the 2025 reanimation.)

One machine fact explains much of the code's texture: the 7094 has 36-bit words and a 6-bit character set, so text is packed **six characters to a word**, with longer words spilling into additional list cells. A "word" of user input is, to this program, one or more 36-bit integers.

District 3: City Hall — the ELIZA main program

After initialization (`INITAS.(0)` must come first — it turns all unused core into SLIP's free space) and script loading — each rule list is hashed by its keyword into the 33-entry table `KEY`, with `KEY(32)` reserved for NONE — the program settles into its major loop, helpfully

bracketed by Weizenbaum's own banner comments:

```
R* * * * * BEGIN MAJOR LOOP
START      TREAD.(MTLIST.(INPUT),0)
```

Read the user's line, then scan it word by word. Three things can happen to a word:

1. **It's a delimiter** — `.` or `,` or, remarkably, `BUT`. If no keyword has been found yet, everything up to the delimiter is discarded and scanning continues; if one has, everything after it is discarded and scanning stops. ELIZA answers one clause, never a whole paragraph.
2. **It's a keyword** — hash it, check the bucket, confirm the full-word match via `TESTS` (next district). Keep only the keyword with the **highest precedence** seen so far (in `KEYWRD` / `IT` / `PREDNC`).
3. **It's nothing** — keep walking.

Then the matching routine tries the chosen keyword's decomposition rules in order (`TRY ... YMATCH ...`), follows any `(=OTHERKEYWORD)` links, selects a reassembly rule, and prints the assembled reply (`HIT ... ASSMBL ... TXTPRN`). Back to START.

★ DON'T MISS

Three places where this 1965 version differs from the system Weizenbaum would describe a year later in the CACM paper (Hay's annotations flag all of these). Remember the direction of time: these aren't errors in the paper — they're a look at ELIZA before it finished growing.

- **BUT is a delimiter** here, alongside comma and period. The paper lists only the latter two — yet, as Hay notes, the paper's own sample conversation behaves as if BUT splits clauses too, so this feature may well have survived into the later version undocumented.
- **There is no keyword stack yet.** The paper describes pushing keywords onto a stack by precedence, with a NEWKEY mechanism to fall back to the next one. This version keeps exactly one keyword — the highest-precedence winner — and nothing else. The stack architecture was evidently a later refinement.
- **Reassembly rules cycle by self-modifying the script.** To use a keyword's reply variants in rotation, the code splices a counter *into the rule's list structure in memory* (`NEWBOT.(1,POINTR)` ... `SUBST.(POINT+1,POINTR)`), incrementing it on each use. The data structure keeps its own minutes.

And savor the script-error exits at the bottom of the file. When the script is malformed or a

linked keyword can't be found, ELIZA doesn't print an error — depending on the counter LIMIT it prints `PLEASE CONTINUE`, `HMMM`, `GO ON`, `PLEASE`, or `I SEE`. Even the crash handler stays in character. A therapist to the end.

District 4: The Customs Office — TESTS

The hash lookup in City Hall only compares the first six-character chunk (one machine word) of a candidate keyword. `TESTS` does the full inspection: it copies every 6-character chunk of the keyword into one array and every chunk of the user's word into another, compares lengths, then compares chunk by chunk. Any mismatch → return 0, no entry.

If the word passes and the rule contains an `=`, `TESTS` performs the substitution on the spot — surgically: `REMOVE.` the chunks of the old word from the user's INPUT list, `NEWTOP.` in the chunks of the replacement, set the packing flags (`MRKNEG` / `MRKPOS`). The user's sentence has now been rewritten in place — MY is already YOUR before any reply is composed. This, not any grammar, is how "you/me" inversion works.

★ DON'T MISS

```
DIMENSION FIRST(5),SECOND(5)
...
T'H ONE, FOR I=0,1, I .G. 100
```

The copy loops run until I exceeds **100**, into arrays dimensioned for **6 words** (36 characters). A keyword longer than 36 characters writes straight past the end of the array into whatever lives next in core. Bounds checking, 1965 style: the bound is a comment in the programmer's head. (A second consequence of the one-word hash probe, noted in the annotations: keywords that share their first six characters can shadow one another — only the first candidate in the bucket gets TESTS'd.)

District 5: The Memory Annex — MYTRAN, MYLIST, and LIMIT

The CACM paper's most quoted piece of misdirection is the throwaway line that, when no keyword is found, a remembered earlier topic may be retrieved "when a certain counting mechanism is in a particular state." Here is the entire mechanism:

```
LIMIT=LIMIT+1
W'R LIMIT .E. 5, LIMIT=1
```

LIMIT counts user inputs 1,2,3,4,1,2,3,4... When the user's sentence contains the script's MEMORY keyword (MY, in DOCTOR), a memory is manufactured immediately — one of the four MEMORY transformations in `MYTRAN` is applied to the input and the result queued on `MYLIST` (e.g. "EARLIER YOU SAID YOUR ..."). Later, if an input contains *no* keyword at all **and LIMIT happens to equal 4** and the queue is non-empty, ELIZA pops the oldest memory and says it — producing the program's most uncanny moments, an apparent return to something you mentioned minutes ago.

★ DON'T MISS

Which of the four MEMORY transformations gets used? The later paper says the choice is random. In this version:

```
I=HASH.(BOT.(INPUT),2)+1
```

— it's the hash of the **last word of your sentence**. There is no randomness anywhere in this listing: given the same script and the same inputs, every conversation, including its eeriest "spontaneous" memories, replays identically. Whether Weizenbaum later added true randomization or simply described the hash as effectively random, the 1965 ghost in the machine is a hash function.

District 6: The Classroom — CHANGE, NEWLST, and the script dump

This district is the part of the 2021 find that drew the most attention, because the published account gives it almost none. The very first routine in the printout is not the chatbot — it's an **editor**.

Type `+` as the first word of any input, and ELIZA suspends the conversation and prints:

```
CHANGE      PRINT COMMENT $PLEASE INSTRUCT ME$
```

It then accepts seven commands — `TYPE`, `SUBST`, `APPEND`, `ADD`, `START`, `RANK`, `DISPLA` — for inspecting and rewriting the live script: print a keyword's rules, substitute a reassembly, append a new one, re-rank a keyword's precedence, display every keyword and the

memory rules. Type `*` and the rest of your line is parsed as a brand-new keyword rule and hashed into `KEY` on the spot (`NEWLST`). And when you end the session with a blank line, ELIZA asks `WHAT IS TO BE THE NUMBER OF THE NEW SCRIPT` and writes the entire revised script back out to tape (`ENDPLA` , via `LPRINT` , which re-prints nested lists with their parentheses restored). Your edits persist; the next person plays your script.

The remaining residents are clerks: `TPRINT` pretty-prints rule lists for the `TYPE` / `DISPLA` commands, and `DOCBCD` converts small numbers to printable characters.

★ DON'T MISS

"PLEASE INSTRUCT ME." Weizenbaum's 1966 paper frames ELIZA as a study in how scripts produce conversation; what the recovered code shows is a working **teaching loop** — converse, notice a bad reply, type `+` , fix the rule, resume conversing — with persistence to tape, in 1965, on a time-sharing console. The ELIZAGEN team has argued this reframes the program: less a canned trick, more an interpreter-plus-editor for conversational competence, where DOCTOR is just the demo cartridge. Whether Weizenbaum intended ELIZA to *learn* (his contemporaneous papers gesture at it) is a live historical question; that he built the apparatus is now simply a fact of the listing.

A Day in the Life: One Sentence, End to End

You type: **MY FATHER IS AFRAID OF EVERYBODY.**

1. **START / TREAD** reads the line into INPUT as packed 6-bit words. LIMIT ticks from 1 to 2.
2. **The scan** walks the words. `MY` hashes into `KEY` ; the bucket yields the MY rule; `TESTS` confirms the full-word match, sees the `=` , and rewrites INPUT in place: `YOUR FATHER IS AFRAID OF EVERYBODY .` MY's precedence is recorded; the scan continues (FATHER may also be a keyword and could outrank it). The final `.` is a delimiter — scanning ends with the highest-precedence keyword in hand.
3. **The Memory Annex**, seeing that the winning keyword is the MEMORY keyword MY, hashes the last word (`EVERYBODY`), picks one of the four MYTRAN rules, and quietly queues "DOES THAT HAVE ANYTHING TO DO WITH THE FACT THAT YOUR FATHER IS AFRAID OF EVERYBODY" (or one of its siblings) on MYLIST. You won't see it now.
4. **MATCH/TRY** runs `YMATCH` of the keyword's decomposition patterns against the rewritten input; `(0 YOUR 0)` -style fragments land in TEST.
5. **The rotation counter** spliced into the rule picks the next unused reassembly; **HIT/ASSMBL** builds the reply from rule plus fragments; `TXTprt` types it — perhaps `TELL ME MORE ABOUT YOUR FAMILY` — with no question mark, because the character

set hasn't got one.

6. **T'O START.** Three inputs later, if you type something keyword-free while LIMIT sits at 4, that queued memory surfaces, and you feel briefly, wrongly, listened to.

Practical Notes for the Independent Traveler

On reading MAD. The apostrophes are abbreviations, and five unlock the file: `W'R` = WHENEVER (if), `T'O` = TRANSFER TO (goto), `T'H label, FOR ...` = THROUGH (a for-loop whose body ends at the label), `O'E / E'L` = OTHERWISE / END OF CONDITIONAL, `F'N` = FUNCTION RETURN. Operators are dotted: `.E.` `.NE.` `.G.` `.L.` `.LE.` Strings are delimited by `$...$`; a `1` in the continuation column splices a statement across cards; the right-hand numbers (000010...) are card sequence numbers, the line numbers of the punched-card era.

On the character set. Six-bit BCD: UPPERCASE ONLY, and **no question mark glyph** — which is why a program famous for answering questions with questions never once prints a "?".

On what's missing. The printout is described by its discoverers as *nearly* complete: the SLIP library itself (MAD plus FAP assembly) and some support functions are not all present, and a handful of passages still carry the transcriber's `???` marks where the reading or intent is uncertain. What survived on paper is the intellectual record — and, since 2025, a runnable one.

On running it. As of 2025 you can. The ELIZA Reanimated project (Rupert Lane, Anthony Hay, Arthur Schwarz, David M. Berry, Jeff Shrager) restored this code on a reconstructed CTSS running on an emulated IBM 7094 — the original chatbot on the original (emulated) operating system, the full stack open source (github.com/rupertl/eliza-ctss). The transcripts, annotations, scans, and the family tree of every ELIZA descendant live at elizagen.org and github.com/jeffshrager/elizagen.org.

What Makes ELIZA Worth Visiting

The legend says ELIZA fooled people; the code shows how little it took. Strip away the SLIP library and the script, and the program that launched a thousand chatbot anxieties is a few hundred lines of scan-hash-match-reassemble, whose deepest "psychological" behaviors reduce to a modulo-4 counter and a hash of your last word. Reading it is a useful calibration: the eliza effect was never in the program. It was, as Weizenbaum spent the rest of his career insisting, in us.

But the visit pays a second way the legend never mentions. The largest single routine in the printout is the script editor. The recovered ELIZA is not a fixed parlor trick; it is an interpreter for a conversational rule language, with a live, persistent editing loop bolted to its front door — a program whose first words, in the right mode, are PLEASE INSTRUCT ME. And because this listing predates the famous paper, you are reading something rarer than a famous program: a famous program *in draft*. The keyword stack isn't built yet; the memory mechanism still runs on a bare hash; the editor sits at the front of the deck as if it were the point of the whole exercise. The 1966 paper shows you ELIZA posed for its portrait. This printout shows you ELIZA on the workbench, mid-assembly — which is, for a visitor who cares how things actually get made, the better view.

////////////////////////////////////

Provenance and Sources

Claims about code behavior come from the annotated transcription of the recovered printout (Hay 2022, in the elizagen.org repository) and are high-confidence where the code is explicit; the transcription itself flags several uncertain passages with `???`, and statements here about `TESTS` edge cases and keyword shadowing follow Hay's bracketed inferences (moderate confidence). The discovery narrative (2021, MIT Institute Archives, "Computer Conversations" folder, CC0 release by the Weizenbaum estate) follows elizagen.org; the dating of the listing as an earlier, pre-publication version of the system described in Weizenbaum (1966) follows the ELIZAGEN team's account, and characterizations of specific differences as "later refinements" are inferences from that dating (moderate confidence). Restoration details follow Lane et al. (2025). The DOCTOR rule excerpts shown follow the script appended to the 1966 CACM paper; the printout's attached script is an earlier variant, so exact rule wordings here are illustrative (moderate confidence).

////////////////////////////////////

References

Works consulted in preparing this guide; not all are cited above.

References

Berry, D. M., & Marino, M. C. (2024). "Reading ELIZA: Critical Code Studies in Action." *Electronic Book Review*.

Ciston, S., Berry, D. M., Hay, A. C., Marino, M. C., Millican, P., Schwarz, A. I., Shrager, J., &

Weil, P. (2026). *Inventing ELIZA: How the first chatbot shaped the future of AI*. The MIT Press. ISBN 9780262052481.

Critical Code Studies Working Group. (2022). “The Original ELIZA in MAD-SLIP (2022 Code Critique).” Thread leaders: Jeff Shrager, David M. Berry, Mark C. Marino, and Jeremy Douglass.

Hay, A. (2022). *Annotated transcription of the original MAD-SLIP ELIZA* (`ELIZA_transcription_annotated_20220216.txt`). In J. Shrager (Ed.), *ELIZAGEN: The Genealogy of ELIZA* repository.

Lane, R., Hay, A., Schwarz, A., Berry, D. M., & Shrager, J. (2025). “ELIZA Reanimated: The world’s first chatbot restored on the world’s first time sharing system.” arXiv:2501.06707.

Lane, R., Hay, A., Schwarz, A., Berry, D. M., & Shrager, J. (2025). “ELIZA Reanimated: Restoring the Mother of All Chatbots to One of the World’s First Time-Sharing Systems.” *IEEE Annals of the History of Computing*, 47(2), 68–76. doi:10.1109/MAHC.2025.3564095.

Shrager, J. (Ed.). *ELIZAGEN: The Genealogy of ELIZA*. Includes discovery account, scans, transcription, and related materials.

Shrager, J. (2024). “ELIZA Reinterpreted: The world’s first chatbot was not intended as a chatbot at all.” arXiv:2406.17650.

Weizenbaum, J. (1963). “Symmetric list processor.” *Communications of the ACM*, 6(9), 524–536. doi:10.1145/367593.367617.

Weizenbaum, J. (1966). “ELIZA—a computer program for the study of natural language communication between man and machine.” *Communications of the ACM*, 9(1), 36–45. doi:10.1145/365153.365168.

Weizenbaum, J. (1967). “Contextual understanding by computers.” *Communications of the ACM*, 10(8), 474–480. doi:10.1145/363534.363545.

Weizenbaum, J. (1976). *Computer Power and Human Reason: From Judgment to Calculation*. San Francisco: W. H. Freeman.

Copyright

Hacker's Guides — guide texts, commentary, and supporting material.

Copyright © 2026 Jeff Shrager (jshrager@gmail.com). All rights reserved.

Permission requests, corrections, and questions: jshrager@gmail.com.

Scope

This copyright covers the original text of the guides and the README — the prose, structure, commentary, and annotations written for this repository:

<https://github.com/jeffshrager/HackersGuides>

It does **not** cover:

- **The historic source code being read.** Quoted program source (and any complete source files included in this repository) remains the property of its respective authors and rights holders, or is in the public domain, as the case may be. Excerpts appear here for purposes of commentary, criticism, and scholarship. Provenance for each program's source is stated in the corresponding guide.
- **Quoted third-party material.** Brief quotations from cited historical accounts and documentation remain the property of their respective rights holders and are used for the same purposes.

Attribution

If you quote or reference a guide, please credit "Hacker's Guides, Jeff Shrager" and link to this repository: <https://github.com/jeffshrager/HackersGuides>